

GRAIN-CAN: Same-ID Local Behavior Features for Cross-Vehicle CAN Intrusion Detection^{*}

Qi'ao Li¹, Simiao Gao², Pengcheng Wang^{1**}, and Yuran Li³

¹ School of Cyber Science and Technology, Beihang University, Beijing 100191, China
{qi_aolee,pcwang}@buaa.edu.cn

² School of Transportation Science and Engineering, Beihang University, Beijing
102206, China
GoesM@buaa.edu.cn

³ Automotive Standardization Research Institute, China Automotive Technology and
Research Center Co., Ltd., Tianjin 300300, China
liyuran@catarc.ac.cn

Abstract. Cross-vehicle CAN intrusion detection is difficult because message distributions and attack patterns can both shift between training and deployment. Raw fixed-window representations further obscure short attack bursts when only a few frames in a long window carry abnormal evidence. This paper proposes GRAIN-CAN, a supervised feature-based CAN IDS pipeline that exposes same-ID local behavior changes before window aggregation. For each CAN frame, GRAIN-CAN maintains a per-ID history state and extracts timing residuals, payload changes, payload statistics, and local ID-behavior features using only past traffic. These frame-level features are aggregated with a fixed pre-test window configuration and classified by a lightweight supervised detector. We evaluate GRAIN-CAN on CT&T Test01–Test04 using attack-positive precision, recall, F1, FNR, FPR, and confusion matrices. AUPR and detection rate at fixed FPR are reported only as supplementary score-based operating metrics. Under the same GradientBoosting classifier, the same-ID residual representation improves F1 over raw-window statistics on the shifted Test02–Test04 settings, with the largest gains on attack-shift and joint-shift tests. Ablations indicate that same-ID timing residuals are the strongest signal in the current CT&T setting.

Keywords: CAN bus · Intrusion detection · Automotive security · Feature pipeline · Cross-vehicle detection

^{*} This work is supported by the National Natural Science Foundation of China (62573030), Zhejiang Provincial Natural Science Foundation of China (LMS26E080008), Beijing-Tianjin-Hebei Basic Research Cooperation Project (E2024210149), and Public Open Project of Automobile Standardization (CATARC-Z-2025-00934).

^{**} Corresponding author.

1 Introduction

Modern vehicles rely on Controller Area Network (CAN) buses to connect electronic control units. The classical CAN protocol was designed for reliability and low-latency broadcast communication rather than message authentication. This design has enabled practical attacks through malicious message injection, spoofing, and remote attack surfaces [1–3]. CAN intrusion detection systems (IDSs) therefore monitor message streams and infer abnormal behavior from the fields that are normally available in traces: timestamps, arbitration IDs, DLCs, and payload bytes.

A detector that performs well on a closed split does not necessarily transfer to another vehicle or another attack family. Cross-vehicle evaluation changes the message distribution: the set of frequent IDs, their periods, payload ranges, and local traffic context can differ across vehicles. Unknown-attack evaluation changes the perturbation mechanism: the injected ID, payload manipulation, timing footprint, or replay pattern may differ from the attack families observed during training. These two shifts interact in the CT&T Test04 setting, where both the test vehicle and attack family are unseen during training [17]. In such a setting, a detector that mainly memorizes absolute CAN-ID frequencies or raw payload ranges can fail for reasons unrelated to attack behavior.

Raw fixed-window representations introduce an additional failure mode. If a short burst affects only a few frames, a long window can contain many normal messages and only a small amount of attack evidence. A sequence model or window-level statistic may still learn to find such evidence, but the burden of preserving a sparse local event is moved to the downstream classifier. This motivates a representation that makes local deviations visible before a window is formed.

GRAIN-CAN follows this design principle. It maintains a compact history state for each CAN ID and converts each frame into local residual features relative to recent messages with the same ID. The residuals include same-ID timing gaps, payload-change magnitudes, coarse payload statistics, and local ID-behavior features. The frame-level features are then aggregated with a fixed pre-test window configuration and passed to a supervised classifier. The classifier is intentionally simple; the contribution is the representation pipeline that exposes same-ID local behavior before aggregation.

The paper makes two contributions.

1. We propose GRAIN-CAN, a supervised feature-based CAN IDS pipeline that extracts same-ID local behavior features before fixed-window aggregation. The design makes short timing and payload-change evidence explicit before it is diluted by long raw windows and reduces reliance on vehicle-specific absolute CAN distributions.
2. We provide a controlled CT&T cross-shift evaluation with conventional IDS metrics, confusion-matrix counts, raw-window representation controls, feature ablations, low-FPR score analysis, and CPU-only deployment-cost measurements. The results isolate representation effects from classifier choice, window length, and metric ambiguity.

2 Background and Design Motivation

2.1 CAN Intrusion Detection Under Cross-Shift

CAN IDS methods have used message timing, ID frequencies, transition patterns, payload dynamics, physical-layer fingerprints, signal-level decoding, learned sequence representations, and graph-based relations [4–7, 11, 14, 21]. Recent automotive IDS work also studies BERT-style message modeling, self-supervised historical learning, multimodal time/signal analysis, explainable signal-aware detection, embedded deployment, and modern sequence backbones [12, 22, 20, 13, 15, 23]. These directions show that CAN IDS is no longer a single-model problem; high-quality systems are usually pipelines with explicit assumptions about input information, training supervision, and deployment constraints.

The CT&T benchmark is useful because it separates vehicle shift and attack shift [17]. Test01 evaluates known vehicles and known attacks. Test02 changes the vehicle while keeping the attack family known. Test03 keeps the vehicle known but changes the attack family. Test04 changes both. This decomposition is important because the same detector can behave differently under vehicle shift and attack shift. ROAD, can-sleuth, and recent comparative work similarly emphasize that IDS claims depend on dataset construction, attack realism, metric definitions, and the availability of comparable predictions or scores [9, 18, 19, 16].

2.2 Loss of Local Evidence in Raw Windows

Many attacks are local in both time and ID space. A spoofing, injection, or replay artifact may affect only a small set of messages while the rest of the window remains benign. Directly feeding a raw sequence to a window-level classifier leaves the model to discover which local change is relevant and to keep that evidence visible after pooling, attention, or summary operations. This can work when the training and test distributions are close, but it becomes less reliable when the test vehicle changes and the model must distinguish attack-induced perturbations from benign shifts in ID frequencies or payload ranges.

Figure 1 illustrates the design motivation. A long raw window can represent a sparse abnormal burst as a small fraction of otherwise normal traffic. A same-ID residual representation turns the same frames into explicit local changes before window aggregation. The point is not that a residual is vehicle-independent. Instead, residual features shift the detector from absolute values toward changes relative to recent same-ID behavior.

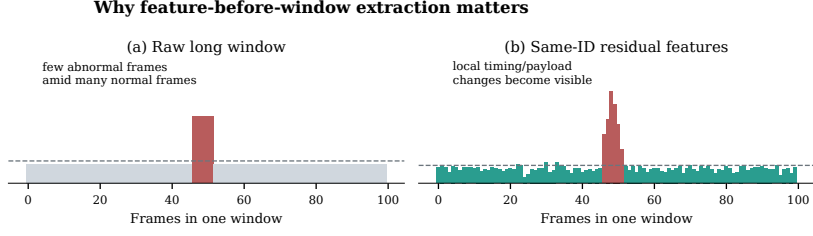


Fig. 1. Feature-before-window extraction exposes sparse local attack evidence before aggregation. A raw long window can dilute a short burst, while same-ID residual features create visible timing or payload-change spikes before the detector sees the window.

2.3 Design Insight

The central insight is to compute local residuals at the frame level before the window is formed. For a frame with arbitration ID id_i , the detector compares the current timestamp and payload with the recent history of the same ID. Timing residuals capture changes in periodicity or burst rate. Payload residuals capture discontinuities in byte-level evolution. Local ID-behavior features capture short concentration changes. These features are then summarized over a window. The fixed window still provides context, but the local evidence is already explicit when aggregation occurs.

This design is narrower than signal-level or self-supervised frameworks. It does not require a DBC file, decoded signals, or a deep backbone. It also does not claim arbitrary zero-day detection. It is a lightweight supervised representation pipeline for settings in which labeled training data are available and the goal is to improve robustness under vehicle and attack shifts.

3 GRAIN-CAN Pipeline

3.1 Overview

GRAIN-CAN has five stages: input stream parsing, per-ID history-state maintenance, same-ID local behavior feature extraction, fixed pre-test window aggregation, and supervised detection. Figure 2 summarizes the pipeline. The detector operates on the raw fields available in CAN traces and does not assume signal-level decoding. All history-dependent quantities use only current and previous frames.



Feature definitions, window size, detector, and threshold are fixed before test-time evaluation.

Fig. 2. GRAIN-CAN detector pipeline. The representation is built before the window is formed: raw frames update a per-ID history state, local residuals are extracted from same-ID history, and the resulting frame features are aggregated for supervised classification.

3.2 Input Stream and Per-ID State

The input stream is ordered by timestamp. A frame is denoted as

$$x_i = (t_i, id_i, dlc_i, b_i), \quad (1)$$

where t_i is the timestamp, id_i is the arbitration ID, dlc_i is the data length, and b_i is the payload byte vector. Labels are used for supervised training and evaluation, not for test-time feature extraction.

For each ID, GRAIN-CAN maintains a history state $H(id)$. In the current implementation this state contains the last timestamp, the last payload, recent count statistics, and payload summary values. When a frame arrives, the extractor reads $H(id_i)$, computes residual features, and then updates the state with the current frame. This read-then-update ordering prevents the current frame from being compared with itself.

3.3 Same-ID Local Behavior Features

The timing residual for frame i is

$$\Delta t_i^{same} = t_i - t_{prev(id_i)}, \quad (2)$$

where $t_{prev(id_i)}$ is the timestamp of the previous frame with the same arbitration ID. If no previous same-ID frame exists, the feature is initialized by a fixed missing-history rule in the feature extractor. This feature captures changes in periodicity, bursts, and replay timing.

The payload-change residual is computed from the current payload and previous same-ID payload. A typical implemented feature is the byte-wise L1 change,

$$\Delta b_i = \sum_{k=1}^8 |b_{i,k} - b_{prev(id_i),k}|, \quad (3)$$

Table 1. GRAIN-CAN pipeline components. The representation modules are fixed before test-time evaluation.

Component	Operation	Control
History state	Stores the last timestamp, last payload, and recent counters for each CAN ID.	Uses past traffic only.
Residual extractor	Computes timing gaps, payload deltas, payload summaries, and local ID behavior.	Reads state before update.
Windowing	Summarizes frame-level residuals into a fixed-length vector.	Window fixed before test.
Detector	Learns a supervised normal/attack boundary and outputs a score or label.	Model and threshold fixed before test.

where $b_{prev(id_i),k}$ is byte k of the previous same-ID payload. Payload summary features include the payload sum, mean, and standard deviation. These summaries do not recover signal semantics; they provide coarse byte-level evidence when DBC information is unavailable.

Local ID-behavior features summarize recent ID concentration and frequency. These features are useful for detecting bursts or local changes in bus composition, but they can also encode vehicle-specific information. The experiments therefore distinguish the residual representation used in the main pipeline from feature sets that include raw or overly vehicle-specific terms.

3.4 Window Construction and Supervised Detector

Frame-level features are aggregated into fixed windows such as $W = 10, 20$, or 100 . The window size is fixed before test-time evaluation. A window is labeled attack if any frame in the window is labeled attack. This rule is common in window-level IDS experiments, but it creates normal-dominated attack windows when attacks are sparse. The role of GRAIN-CAN is to make local evidence visible before this label is assigned to the aggregate sample.

The repository experiments instantiate GRAIN-CAN with GradientBoosting-style classifiers. The classifier is not the methodological novelty. It learns a supervised decision boundary over features produced by the same-ID residual pipeline. When a classifier exposes a continuous score, the paper reports AUPR and detection rate at fixed FPR only as supplementary score-based operating evidence.

3.5 Training and Inference Protocol

Training. Given a fixed window length W , GRAIN-CAN initializes an empty per-ID history table and processes the training stream in timestamp order. For each frame, it reads the current same-ID state, computes timing residuals, payload residuals, payload summaries, and local ID-behavior features, updates the state, and appends the feature vector to the current window. After every W frames,

the frame features are aggregated into a window vector. The supervised detector is then trained on window vectors and normal/attack labels. If a score threshold is required, it is selected on validation data or by a pre-declared rule.

Inference. At test time, feature definitions, encoders, window length, detector, and threshold are fixed. The test stream is processed once in timestamp order. Feature extraction uses only current and previous frames. Test labels are used only after inference to compute metrics. The reported W10, W20, and W100 rows are sensitivity configurations, not test-time hyperparameter selection.

4 Related Work

4.1 Rule, Statistical, and Timing CAN IDS

Early CAN IDS work used message intervals, ID frequencies, entropy, and transition statistics because many injection attacks disturb periodic traffic [4, 5, 10]. Such detectors are attractive because they are transparent and lightweight. Their limitation is that a single statistic or manually selected threshold may not capture attacks that preserve timing or that combine multiple weak signals. GRAIN-CAN keeps the interpretable evidence used in this line of work but uses it as supervised features rather than as a single threshold rule.

4.2 Deep Sequence and Representation Learning

Learning-based CAN IDS methods use raw message sequences, payload bits, tokenized frames, autoencoders, graph encoders, and recent sequence models [8, 12, 14, 23]. Their advantage is capacity: they can learn temporal dependencies that are difficult to hand-code. Their risk is that the learned representation may entangle attack evidence with vehicle-specific ID and payload distributions. The raw Transformer baseline in this paper illustrates this tradeoff: it is strong in the known Test01 setting but degrades under vehicle or attack shifts. GRAIN-CAN takes a different route by making a small set of same-ID residuals explicit before the classifier sees a window.

4.3 Signal-Level, Multimodal, and Explainable Frameworks

Recent framework-level CAN IDSs often become stronger by using more structured information. CANShield parses signal-level time series and applies multi-scale autoencoder analyzers [11]. CANival combines a time-interval likelihood analyzer with signal-based analysis [20]. X-CANIDS uses a CAN database to convert payloads into human-understandable signals and to support explanation [13]. These methods are attractive when signal semantics are available, and they are better suited to attacks that preserve ID frequency but alter physical signals. GRAIN-CAN deliberately targets a lower-information setting: timestamp, ID, DLC, and payload fields only. Its benefit is portability across traces without DBC-style signal decoding; its limitation is weaker semantic interpretation.

Table 2. CT&T cross-shift settings used in the main evaluation. Positive rate is computed at the evaluated window or sample unit used by the existing result package.

Setting	Vehicle shift	Attack shift	Frames	Pos. rate	Role
Test01	known	known	16,355,810	0.007	Closed-shift base-line
Test02	unknown	known	17,101,057	0.012	Vehicle-shift stress test
Test03	known	unknown	19,288,415	0.004	Attack-shift stress test
Test04	unknown	unknown	23,873,695	0.003	Joint vehicle/attack-shift stress test

4.4 Historical, Self-Supervised, and Unknown-Attack IDS

Unknown-attack IDS is often more naturally served by unsupervised or self-supervised training because labels for future attacks are unavailable. CANet and HistCAN learn benign or historical traffic representations without relying on labeled attack families [8, 22]. Such methods have a stronger claim to zero-day deployment than a supervised classifier. GRAIN-CAN is therefore framed more narrowly: when labeled normal/attack data exist, it tests whether same-ID local residuals improve supervised cross-shift detection. It does not replace benign-only or signal-level methods; it offers a lightweight representation that can be combined with those directions in future work.

4.5 Benchmarking and Comparability

Recent surveys and benchmark studies emphasize that CAN IDS comparisons depend on the workload, attack model, feature type, label unit, and metric definition [16, 18, 19]. This paper therefore reports conventional attack-positive metrics and confusion counts as primary evidence. AUPR, AUROC, and detection rate at fixed FPR are useful for score-producing detectors, but they are not used to compare against prior work unless comparable predictions or scores are available.

5 Experimental Setup

5.1 Datasets and Cross-Shift Settings

The main evaluation uses CT&T set01. Table 2 summarizes the four settings. They form a categorical 2×2 shift decomposition rather than a time series. Test02 isolates vehicle shift, Test03 isolates attack-family shift, and Test04 combines both.

5.2 Compared Pipelines

The main comparison includes representative reproduced pipelines rather than every historical row in the repository. GB-sample and MLP use public-style sample features. Raw-Trans is the existing raw fixed-window Transformer row. Raw-window+GB and GRAIN-CAN+GB form the key representation-controlled comparison because they use the same classifier family and W100 window length. Simple timing or payload thresholds are retained only as diagnostic checks and are not treated as recent state-of-the-art competitors.

Table 3. IDS pipelines compared in the evaluation. Raw-GB and GRAIN-GB use the same classifier family; they differ in representation before aggregation.

Detector	Input representation	Model	Role
GB-sample	Public-style sample features	GB	Reproduced classical baseline
MLP	Public-style sample features	MLP	Neural classical baseline
Raw-Trans	Raw W100 frame sequence	Trans.	Raw-window neural baseline
Raw-GB	Raw W100 window statistics	GB	Same-classifier control
GRAIN-GB	Same-ID residual W100 features	GB	Proposed detector pipeline

5.3 Metrics and Threshold Rules

Attack is the positive class. We report precision, recall, F1, FNR, and FPR as primary IDS metrics:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}. \quad (4)$$

where P is precision, R is recall or detection rate, TP is the number of detected attacks, FP is the number of false alarms, and FN is the number of missed attacks.

We also report false-negative and false-positive rates:

$$\text{FNR} = \frac{FN}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}. \quad (5)$$

where TN is the number of benign windows correctly classified as normal. FNR measures missed attacks, and FPR measures false alarms on benign traffic.

For score-producing models, supplementary operating analysis uses detection rate under a false-alarm budget:

$$\text{TPR}(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)} \quad \text{s.t.} \quad \text{FPR}(\tau) \leq \alpha. \quad (6)$$

where τ is the score threshold and α is the allowed FPR budget. These operating points are diagnostic score-based metrics. They are not compared to prior papers without access to the corresponding scores.

5.4 Implementation Details

The repository result package reports W100 as the main fixed-window configuration for the representation-controlled comparison. The same-classifier experiment uses GradientBoosting with validation-F1 threshold selection. Raw-window+GB and GRAIN-CAN+GB share the same classifier family, train/test split, window length, and metric definitions, so their comparison isolates representation as much as the available artifacts allow. The added efficiency measurements are CPU-only and were repeated three times on the first two CT&T Test04 files; feature extraction time includes CSV parsing and residual construction. These measurements quantify current implementation cost, not optimized embedded performance. Classifier sensitivity and rule-threshold diagnostics are treated as supplementary checks rather than main comparisons.

6 Evaluation

6.1 Cross-Shift Detection Results

Table 4 summarizes representative CT&T results. The main pattern is not that one detector wins every setting. Raw-Trans performs strongly in the closed Test01 setting, but its F1 collapses on Test02–Test04 because its high recall comes with excessive false alarms or severe misses. GRAIN-CAN+GB is less dominant in Test01 but remains substantially more stable under vehicle and attack shifts.

6.2 Effect of Same-ID Residual Representation

The central control experiment compares raw-window statistics and same-ID residual features with the same GradientBoosting classifier and W100 window length. This isolates representation from classifier choice. Table 5 shows that raw-window features are slightly better in Test01, but GRAIN-CAN features improve F1 in all shifted settings. The gain is small in Test02, large in Test03, and large in Test04.

Table 4. Representative CT&T results with attack as the positive class. Values are rounded to three decimals.

Setting	Detector	P	R	F1	FNR	FPR
Test01	GB-sample	0.860	0.980	0.916	0.020	0.001
Test01	Raw-Trans	1.000	0.931	0.964	0.069	0.000
Test01	GRAIN-CAN+GB	0.999	0.892	0.942	0.108	0.000
Test02	GB-sample	0.931	0.998	0.964	0.002	0.001
Test02	Raw-Trans	0.087	1.000	0.160	0.000	0.133
Test02	GRAIN-CAN+GB	0.984	0.931	0.956	0.070	0.000
Test03	GB-sample	0.719	0.629	0.671	0.371	0.001
Test03	Raw-Trans	1.000	0.013	0.025	0.987	0.000
Test03	GRAIN-CAN+GB	1.000	0.662	0.797	0.338	0.000
Test04	GB-sample	0.707	0.447	0.548	0.553	0.000
Test04	Raw-Trans	0.011	0.980	0.022	0.020	0.229
Test04	GRAIN-CAN+GB	0.938	0.519	0.668	0.481	0.000

Table 5. Representation-controlled comparison using the same GradientBoosting classifier and W100 windows.

Setting	Representation	P	R	F1	FNR	FPR
Test01	Raw-window statistics	0.999	0.938	0.968	0.062	0.000
Test01	Same-ID residuals	0.999	0.892	0.942	0.108	0.000
Test02	Raw-window statistics	1.000	0.895	0.944	0.105	0.000
Test02	Same-ID residuals	0.985	0.930	0.957	0.070	0.001
Test03	Raw-window statistics	1.000	0.072	0.134	0.928	0.000
Test03	Same-ID residuals	1.000	0.661	0.796	0.339	0.000
Test04	Raw-window statistics	0.694	0.140	0.233	0.860	0.001
Test04	Same-ID residuals	0.958	0.633	0.762	0.367	0.000

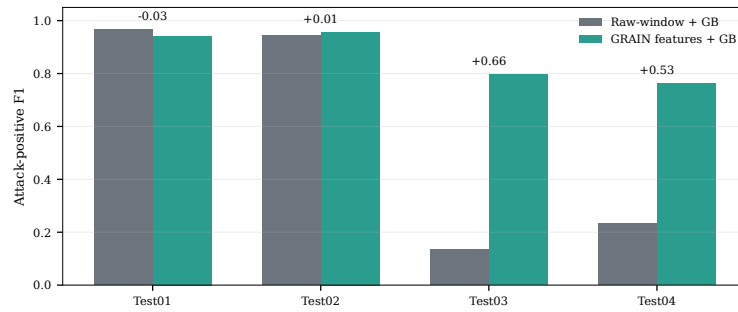
**Fig. 3.** Representation-controlled comparison. With the same GradientBoosting classifier, same-ID residual features are not better in the closed Test01 setting, but they improve F1 under the three shifted settings. Numbers above each group denote the F1 change of GRAIN-CAN over raw-window statistics.

Figure 4 visualizes the same result as a shift matrix. This avoids treating Test01–Test04 as a time series. The largest gains occur when attack families are unknown, and the joint-shift Test04 also benefits substantially.

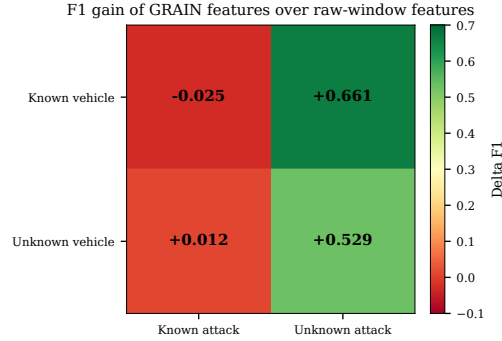


Fig. 4. F1 gain of same-ID residual features over raw-window statistics in the CT&T vehicle-shift by attack-shift matrix. The categorical matrix is more appropriate than a line plot because Test01–Test04 are not temporal samples.

6.3 Contribution of Local Behavior Features

The feature ablation indicates that the main signal in the current CT&T setup is timing residual behavior. On Test04, timing-only features reach F1 0.673, while the full feature set that includes vehicle-specific raw ID and payload terms is weaker under the ablation protocol. Removing CAN-ID behavior improves over the full feature set in the joint-shift setting. This result narrows the method claim: GRAIN-CAN should be viewed primarily as a residual representation pipeline, and the most reliable component in the current evidence is same-ID timing. Payload and ID features may help in some settings, but they should not be overclaimed without additional signal-level evidence.

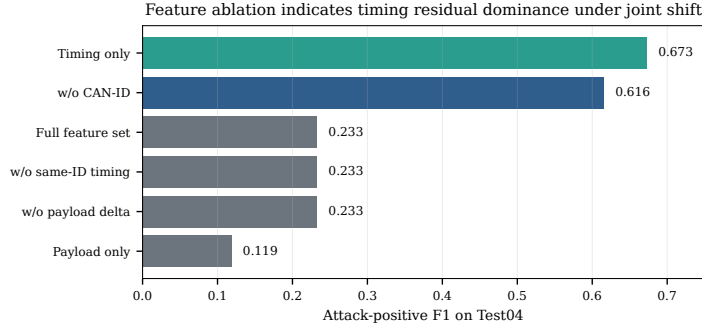


Fig. 5. Feature ablation on Test04. Same-ID timing residuals are the dominant local-behavior signal under joint shift, while raw ID and payload-heavy variants can be less robust.

6.4 Low-False-Alarm Operating Points

Score-based operating metrics are supplementary. On Test04, GRAIN-CAN+GB has AUPR 0.790 and AUROC 0.904, while Raw-window+GB has AUPR 0.107 and AUROC 0.570. At an FPR budget of 10^{-3} , GRAIN-CAN detects 80.5% of attack windows in the diagnostic operating curve, compared with 14.0% for Raw-window+GB. Figure 6 reports several FPR budgets. These values should be read as score-based operating evidence, not as direct comparisons with prior papers that do not expose scores.

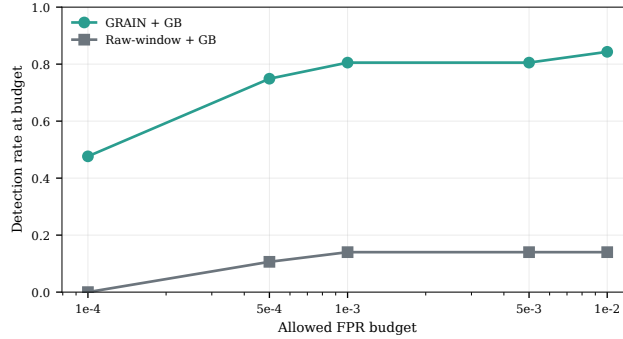


Fig. 6. Supplementary low-FPR operating points on Test04. Same-ID residual features provide substantially higher detection rate than raw-window statistics at matched FPR budgets.

Table 6. CPU-only deployment-cost measurement on the first two CT&T Test04 files. The measurement includes stream parsing and feature construction and is repeated three times.

Pipeline	Frames/s	Windows/s	End-to-end ms	Model MB
Raw-window+GB	98,269	983	24,497	0.025
GRAIN-CAN+GB	97,000	970	24,818	0.025

6.5 Deployment Cost

The CPU-only measurement shows that the current implementation processes about 97k frames per second end-to-end for GRAIN-CAN+GB on the measured Test04 files. Raw-window+GB reaches about 98k frames per second. The difference is small because both pipelines are dominated by CSV parsing and feature construction rather than classifier inference. The model size is about 0.025 MB for both GradientBoosting variants. This supports a lightweight interpretation of GRAIN-CAN, but it does not substitute for optimized embedded measurements.

6.6 Classifier Sensitivity

A supplementary classifier check fixes the GRAIN-W100 representation and varies the classifier. The residual representation remains effective with logistic regression and random forest variants, indicating that the main evidence is not tied to one GradientBoosting implementation. Detailed rows are kept in the experiment package rather than the main text.

7 Discussion

7.1 Supported Claims

The strongest supported claim is representation-level. Same-ID residual features before aggregation improve over raw-window statistics under the same classifier in Test02–Test04. The data also show that raw-window neural detection is not uniformly weak: it is strong in Test01 but unstable when vehicle or attack shifts appear. This pattern supports the paper’s design motivation without overstating universal superiority.

The ablation results refine the mechanism. Timing residuals are the dominant signal in the current CT&T setup. This suggests that the present performance comes mainly from exposing timing disruptions before window aggregation. Payload and ID features should be treated as modular evidence sources rather than guaranteed contributors under every shift.

7.2 Failure Modes and Boundaries

GRAIN-CAN is supervised. It requires normal/attack labels for training and does not learn a benign-only model. It may miss attacks that preserve same-ID

timing, payload continuity, and local ID behavior. It also reduces, but does not remove, dependence on vehicle-specific distributions. Raw ID and payload features can even hurt under joint shift if they encode vehicle-specific patterns. Future versions should consider train-only normalization, signal-level payload features, benign-only pretraining, and online adaptation.

7.3 Comparability to Prior Work

Prior reported aggregate F1 is not directly comparable with attack-positive F1 unless the positive class, averaging rule, threshold rule, and confusion matrix are known. Similarly, AUPR and detection rate at fixed FPR require score outputs. This paper therefore compares locally reproduced rows under common definitions and treats score-based operating metrics as supplementary evidence. The paper does not claim that earlier CT&T or ROAD results are wrong; it only separates what can be recomputed from what is externally reported.

8 Conclusion

GRAIN-CAN is a supervised feature pipeline for CAN IDS that makes same-ID local behavior explicit before window aggregation. The representation-controlled experiments show that this design improves over raw-window statistics under vehicle and attack shifts, especially in Test03 and Test04. The added deployment measurement indicates that the current implementation is lightweight on CPU, while ablations show that timing residuals carry the strongest signal. The method remains limited by supervised training assumptions and by attacks that preserve timing and payload continuity. Future work should combine same-ID residual features with self-supervised training, signal-level features, DBC-aware variants, and real-time in-vehicle deployment studies.

References

1. Koscher, K., et al.: Experimental security analysis of a modern automobile. In: IEEE Symposium on Security and Privacy, pp. 447–462 (2010)
2. Checkoway, S., et al.: Comprehensive experimental analyses of automotive attack surfaces. In: USENIX Security Symposium (2011)
3. Miller, C., Valasek, C.: Remote exploitation of an unaltered passenger vehicle. Black Hat USA (2015)
4. Song, H.M., Kim, H.R., Kim, H.K.: Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network. In: ICOIN, pp. 63–68 (2016)
5. Taylor, A., Leblanc, S., Japkowicz, N.: Frequency-based anomaly detection for the automotive CAN bus. In: WCICSS, pp. 45–49 (2015)
6. Cho, K.T., Shin, K.G.: Fingerprinting electronic control units for vehicle intrusion detection. In: USENIX Security Symposium, pp. 911–927 (2016)
7. Cho, K.T., Shin, K.G.: Viden: Attacker identification on in-vehicle networks. In: ACM CCS, pp. 1109–1123 (2017)

8. Hanselmann, M., Strauss, T., Dormann, K., Ulmer, H.: CANet: An unsupervised intrusion detection system for high dimensional CAN bus data. *arXiv:1906.02492* (2019)
9. Verma, M.E., Bridges, R.A., Hollifield, S.C., Iannacone, M.D., Moriano, P.: ROAD: The real ORNL automotive dynamometer controller area network intrusion detection dataset. *arXiv:2012.14600* (2020)
10. Blevins, D.H., Moriano, P., Bridges, R.A., Verma, M.E., Iannacone, M.D., Hollifield, S.C.: Time-based CAN intrusion detection benchmark. *arXiv:2101.05781* (2021)
11. Shahriar, M.H., Xiao, Y., Moriano, P., Lou, W., Hou, Y.T.: CANShield: Deep learning-based intrusion detection framework for controller area networks at the signal-level. *arXiv:2205.01306* (2022)
12. Alkhatib, N., Mushtaq, M., Ghauch, H., Danger, J.L.: CAN-BERT do it? Controller area network intrusion detection system based on BERT language model. *arXiv:2210.09439* (2022)
13. Jeong, S., Lee, S., Lee, H., Kim, H.K.: X-CANIDS: Signal-aware explainable intrusion detection system for controller area network-based in-vehicle network. *IEEE Transactions on Vehicular Technology* (2024)
14. Wang, K., Jiang, Q., Wang, B., Zhang, Y., Wu, Y.: Effective in-vehicle intrusion detection via multi-view statistical graph learning on CAN messages. *arXiv:2311.07056* (2023)
15. Khandelwal, S., Shanker, S.: A lightweight multi-attack CAN intrusion detection system on hybrid FPGAs. *arXiv:2401.10689* (2024)
16. Sharmin, S., Mansor, H., Abdul Kadir, A.F., Aziz, N.A.: Benchmarking frameworks and comparative studies of controller area network intrusion detection systems: A review. *arXiv:2402.06904* (2024)
17. Lampe, B., Meng, W.: can-train-and-test: A curated CAN dataset for automotive intrusion detection. *Computers & Security* 140, 103777 (2024)
18. Kidmose, E., et al.: can-sleuth: Sleuthing out capabilities, limitations, and performance impacts of automotive intrusion detection datasets. In: *EICC* (2024)
19. Guerra, L., Xu, L., Bellavista, P., Chapuis, T., Duc, G., Mozharovskiy, P., Nguyen, V.T.: AI-driven intrusion detection systems on the ROAD dataset: A comparative analysis for automotive CAN. *arXiv:2408.17235* (2024)
20. Kang, H., Vo, T., Kim, H.K., Hong, J.B.: CANival: A multimodal approach to intrusion detection on the vehicle CAN bus. *Vehicular Communications* 49, 100845 (2024)
21. Song, J., et al.: DGIDS: Dynamic graph-based intrusion detection system for CAN. *Computers & Security* 145, 104076 (2024)
22. Zhuo, S., et al.: HistCAN: A real-time CAN IDS with enhanced historical traffic learning capability. In: *NDSS VehicleSec* (2024)
23. Liu, Q., Song, R., Cui, L., Zhang, H., Sun, Y., Sun, L.: MIDS: Detecting stealthy masquerade and tampering attacks on CAN bus via bidirectional Mamba. *arXiv:2606.18599* (2026)
24. Saito, T., Rehmsmeier, M.: The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE* 10(3), e0118432 (2015)